
Bases de Datos 2

— Repaso, Integridad y Transacciones —

Repaso

Base de datos

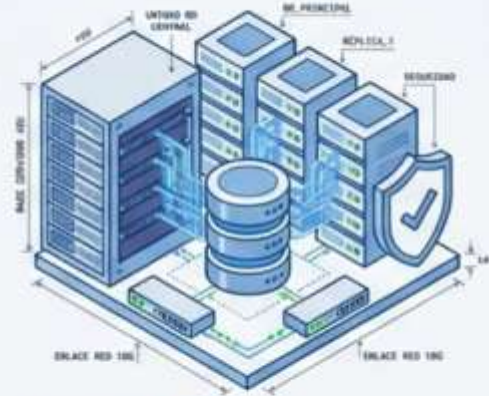
Sistemas de Archivos (El Problema)



- Carencia de enfoque metodológico
- Redundancia de datos
- Inconsistencias
- Conflictos de acceso concurrente

LÍNEA DE DIVISIÓN: ANTES vs. DESPUÉS

SGBD (La Solución)



- **Independencia** programas/datos
- **Integridad** garantizada
- **Seguridad** robusta
- Administración centralizada



Definición Clave:

Una **Base de Datos (BD)** es una colección compartida de datos lógicamente relacionados junto con su descripción (**metadatos**), diseñada para satisfacer las necesidades de una organización.

Diseño conceptual - Diagrama Entidad-Relación

Descripción semántica del “mimimundo” tras la recopilación de requisitos.

- Entidades: Objetos distinguibles en el mundo real.

- Fuerte: Clave propia

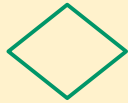


- Débil: Dependencia existencial



- Atributos: Propiedades que describen a las entidades

- Relaciones



- Cardinalidades:

- 1 a 1



1 1

- 1 a muchos

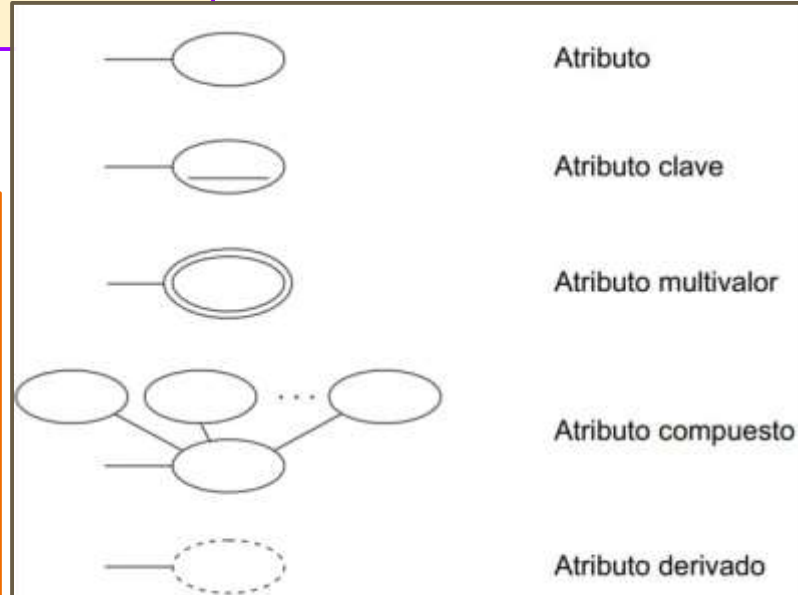


1 N

- muchos a muchos



N M



Atributo

Atributo clave

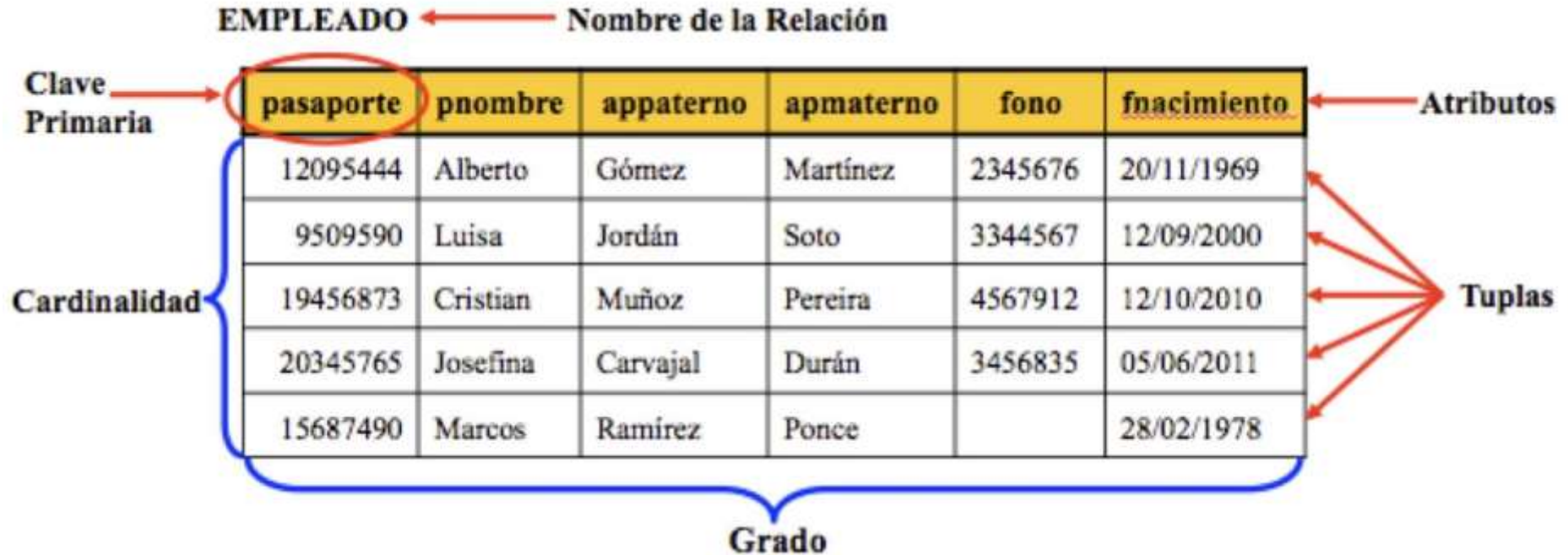
Atributo multivalor

Atributo compuesto

Atributo derivado

Estructura Lógica - Modelo Relacional

Modelo lógico basado en la teoría matemática de conjuntos.



Estructura clara y rígida para facilitar el almacenamiento físico y consulta lógica.

Integridad de los datos

Las Restricciones de Integridad:

- Informan al SGBD de **restricciones del mundo real**.
- Así, el SGBD evita configuraciones de datos imposibles.
- Aumentan la capacidad expresiva del modelo relacional.
- Son específicas de cada BD particular, pero el Modelo Relacional incluye **características generales de integridad** importantes y necesarias en **toda** BD
 - **Reglas de integridad de las entidades (clave primaria)**
 - **Reglas de integridad referencial (clave foránea)**

Integridad de Datos - SuperClave de una Relación

Superclave (SK)

Es cualquier conjunto de atributos que permite identificar de manera única cada tupla (fila) en una relación (tabla).

Restricción de unicidad: No puede haber dos filas con la misma combinación de valores en esa superclave.

Ejemplo: ALUMNOS(cuil, legajo, nombre, apellido)

Superclaves:

- {cuil} (alcanza para identificar a cada alumno).
- {legajo} (también alcanza).
- {cuil, nombre} (sigue garantizando unicidad, aunque “nombre” sobra).
- {cuil, apellido} (ídem, aunque redundante).
- {cuil, legajo, nombre, apellido} (todo el conjunto también es superclave).

Integridad de Datos - Claves de una Relación

Clave candidata (CK)

Es una **superclave mínima**, es decir, **sin atributos redundantes**.

Restricción de irreductibilidad: Si se quita un atributo, ya no cumple la unicidad.

Cada clave es una restricción de integridad.

Ejemplo: ALUMNOS(cuil, legajo, nombre, apellido)

{cuil} y {legajo} son claves candidatas.

{cuil, nombre} ✗ no es candidata, porque “nombre” sobra.

Integridad de Datos - Clave Primaria de una Relación

Clave Primaria (PK)

Es la clave candidata seleccionada para identificar cada tupla de forma única.

Ejemplo: ALUMNOS(cuil, legajo, nombre, apellido)

si elegimos {cuil} como **PK** → identificamos de forma única a cada alumno.

Regla de integridad de entidad

Ningún atributo de la clave primaria puede tener valores nulos.

Ejemplo: no puede existir un alumno sin cuil si este es la PK.

Integridad de Datos - Clave Externa o Foránea

Clave externa o Foránea (FK)

Conjunto de atributos de una relación que hace referencia a la clave primaria de otra relación.

Ejemplos:

- PELICULAS (título, género, duración, *director_dni*, ...)
- DIRECTORES (dni, nombre, nacionalidad, ...)

-
- EMPLEADOS (codEmp, nombre, *codJefe*, nss, ...)

-
- LIBROS (título, isbn, *autor_dni*, editorial, edición, año, ...)
 - ESCRITORES (dni, nombre, ...)
 - ARTICULOS (título, tema, *autor_dni*, revista, página, ...)

Integridad de Datos - Clave Externa o Foránea

Cada componente de una FK debe estar definido sobre el mismo dominio que el correspondiente atributo de la PK a la que referencia.

- Ejemplo
 - Departamentos(id_depto, nombre)
 - Empleados(id_empleado, nombre, *id_depto*)
 - Donde Empleados.id_depto es FK que referencia a Departamentos.id_depto.
Ambos atributos (id_depto) deben pertenecer al mismo dominio, por ejemplo Enteros positivos.

El uso de **Claves Foráneas** facilita:

- Eliminación de redundancia: integridad entre archivos
- Mecanismo del modelo relacional de datos para establecer **vínculos entre relaciones**.

Integridad de Datos - Clave Externa o Foránea

Regla de integridad referencial

Todo valor no nulo de una clave foránea (FK) debe coincidir con un valor en la relación referenciada (PK)

Nota: El Modelo Relacional permite nulo como valor de clave foránea cuando la relación no es obligatoria.

Integridad de Datos - Integridad Referencial

Clave primaria de la relación Empleados

Relación referida

Empleados

DNI	Nombre	Apellidos	Direccion	Telefono	f_Nacimiento
23456789	Pedro	Ramírez	Zapiola 5	548456856	11/5/2000
12345678	María	González	Av.Alem 12	548965123	3/3/1995

Relación referencial

Clave foránea en la relación Puestos

Clave primaria de la relación Puestos

Puestos

id_caja	descripcion	dni_empleado
1	Caja número 1 del sector A	23456789
2	Caja número 2 del sector A	12345678
3	Caja número 1 del sector B	

El DNI de la tabla Puestos debe existir previamente en la tabla Empleados.

- La clave ajena y la clave primaria correspondiente deben definirse sobre el mismo dominio
- La clave ajena no tiene porqué formar parte de la clave primaria de la relación que la contiene
- Las claves ajenas pueden admitir nulos, las claves primarias no

Mantenimiento de la Integridad Referencial

Las operaciones que no satisfacen -violan- la integridad Referencial, dejan la BD en un estado incorrecto.

Ejemplo de un hotel:

- ¿Qué pasaría si se eliminara la tupla (501, doble, ...) en habitaciones?
- ¿Y si se eliminara la tupla (100, doble, ...)?
- ¿Y si se registrara la ocupación de la habitación 900?

ocupaciones	codCliente	habitacion	...
	123	100	
	25	420	
	56	115	
	12	100	
	95	304	

habitaciones	numHabitacion	tipo	...
	115	individual	
	420	individual	
	100	doble	
	304	doble	
	405	individual	
	501	doble	

Mantenimiento de la Integridad Referencial

¿Cómo evita el SGBD esos estados incorrectos?

El SGBD puede:

- **Rechazar** toda operación que pueda provocar un estado ilegal (que no garantice la integridad referencial), ó
- **Aceptar** (y ejecutar) tales operaciones, **pero realizando acciones** que restauren la integridad de los datos.

El diseñador de la BD puede especificar al SGBD **Acciones de Mantenimiento de la Integridad Referencial** para que la BD SIEMPRE alcance un estado final legal.

Mantenimiento de la Integridad Referencial

Ejemplo: **Borrar un valor que es clave primaria en una relación y ajena en otra**

Eliminar la tupla $t=(100, D, \dots)$ de habitaciones

Acciones posibles:

1. **Rechazar** la operación (Acción por defecto): Sólo permite borrar t si ninguna otra tupla hace referencia a t .
2. **Cascada**: propagar la eliminación a las tuplas que la referencian mediante la clave ajena y luego se borra la tupla en cuestión.
 - a. 1° borrar todas las tuplas de **ocupaciones** que referencian a t
 - b. 2° eliminar t
3. **Establecer nulos**: se pone a nulo (null) la clave ajena en toda tupla de ocupaciones que referencia a t y luego se borra la tupla t . **Esto sólo si la FK de ocupaciones acepta nulos!**

Resumiendo, el SGBD se encarga de...

- **Comprobar las claves candidatas** (primaria y alternativas):
 - No existen dos tuplas distintas con igual valor para una clave.
- **Comprobar la restricción de Integridad de Entidad:**
 - Ningún atributo componente de una clave primaria contiene nulo.
- **Comprobar la restricción de Integridad Referencial:**
 - El valor de la clave foránea en cualquier tupla, o es nulo, o coincide con un valor de clave primaria de alguna tupla en la relación referenciada.
 - y tambien se encarga de **mantenerla** frente a operaciones que puedan violar la integridad.



**What is needed for a
good relationship?**



Primary Keys

Pasaje MER a MR - Resumen

Transformación de Entidades:

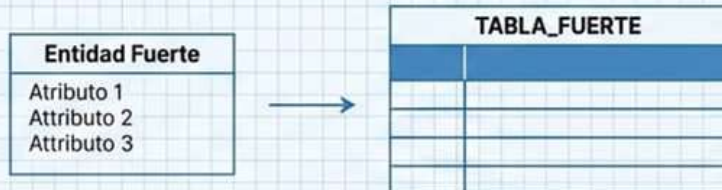
- Se crea una tabla relacional por cada entidad
- Todo atributo de la entidad se transforma en un atributo de la tabla relacional
- El identificador de la entidad se transforma en la clave primaria de la tabla relacional

Transformación de Relaciones:

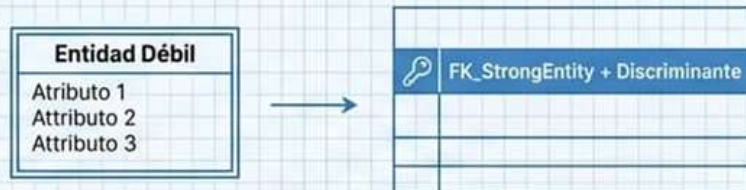
- En las interrelaciones unarias o binarias con cardinalidad 1:N se adicionan los atributos identificatorios de la entidad del lado '1' a la entidad del lado 'N'.
- Las relaciones binarias N:N y n-arias se transforman en una nueva tabla relacional con los atributos claves de las entidades participantes en la relación y los atributos propios de la relación (si los tiene)

Mapeo: Del Diagrama MER a Tablas Relacionales

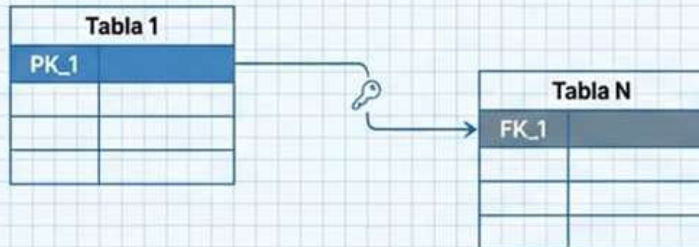
- **1. Entidades Fuertes** Conversión directa a tabla. Atributos simples se vuelven columnas.



- **2. Entidades Débiles** Tabla con PK compuesta: PK de entidad fuerte + discriminante.



- **3. Relaciones 1:N** Propagar PK del lado "1" como FK en el lado "N".



- **4. Relaciones N:M** Crear nueva tabla intermedia con ambas PKs como Claves Foráneas.



Normalización (1) - Definiciones

Objetivo: **eliminar redundancias y evitar anomalías**

- **Dependencia Funcional**

Se dice que un atributo B de una relación R depende funcionalmente del atributo A de R , si para todo instante de tiempo, cada valor de A no tiene más de un atributo asociado de B en la relación R .

La forma de denotar que B depende funcionalmente de A es: $A \twoheadrightarrow B$

- **Clave o llave de la relación**

Se dice que un conjunto de atributos A en una relación R constituyen una *clave o llave* para dicha relación si todos los atributos de R dependen funcionalmente de A , esto es $A \twoheadrightarrow R$, y no existe ningún subconjunto A' , $A' \subset A$ tal que A' determine funcionalmente a R .

Normalización (2) - Definiciones

- Atributos primos y no primos

Se dice que un atributo A de una relación R es '*no primo*' si no forma parte de ninguno de los posibles conjuntos claves para R. Caso contrario A es un atributo '*primo*'.

- Dependencia funcional total

Un atributo o conjunto de atributos B de una relación R se dice que '**depende funcionalmente en forma total**' de otro conjunto de atributos A de la relación si B depende funcionalmente de la totalidad de A, pero de ningún subconjunto de A.

Primera Forma Normal

Una tabla está en **Primera Forma Normal (1FN)** cuando:

1. **Todos los atributos tienen valores atómicos:** no hay conjuntos, listas ni registros anidados; cada celda contiene un solo valor indivisible.
2. **No existen grupos repetidos:** cada fila es única y no hay columnas que representen repeticiones de un mismo concepto (por ejemplo, **teléfono1**, **teléfono2**, etc.).
3. **Cada tupla es identificable de forma única** mediante una **clave primaria**.

Segunda Forma Normal - Eliminar dependencias parciales

Una relación R está en 2FN si:

- I. Está en 1FN
- II. Todo atributo *no primo* de R *depende funcionalmente en forma total* de cada clave candidata de R.

Pasos a seguir para pasar a la 2FN:

1. Identificar la clave primaria de la tabla.
2. Detectar los atributos que dependen solo de una parte de la clave.
3. Separar esos atributos en una nueva tabla, usando como clave primaria ese subconjunto de la clave de la tabla original.
4. Mantener una relación entre ambas tablas mediante la clave que comparten.

Tercer Forma Normal - Eliminar dependencias transitivas

Una relación R está en 3FN. cuando:

- I. Está en 2FN
- II. No existen dependencias transitivas

Existe una '**dependencia transitiva**' cuando un atributo (o conjunto de atributos) que no forma parte de la clave depende de otro atributo (o conjunto de atributos) que tampoco forman parte de la clave.

Ejemplo de normalización de base de datos

Como ejemplo de normalización, se propone una base de datos para almacenar información sobre estudiantes en una escuela. Esta es una tabla simple sin normalizar con la información de los estudiantes:

ID	Nombre	Edad	Grado	Profesor/a	Teléfono
1	Juan	15	10	Sr. Martínez	658253524
2	María	14	9	Sra. López	665372145
3	Carlos	16	11	Sra. Pérez	687253940
4	Ana	15	10	Sr. García	698321475

Primera forma normal (1FN)

En la primera forma normal, es preciso cerciorarse de que cada columna tenga valores atómicos y del mismo tipo de datos. En este caso, todos los valores ya son atómicos y del mismo tipo, así que esa tabla ya se encuentra en 1FN.

ID	Nombre	Edad	Grado	Profesor/a	Teléfono
1	Juan	15	10	Sr. Martínez	658253524
2	María	14	9	Sra. López	665372145
3	Carlos	16	11	Sra. Pérez	687253940
4	Ana	15	10	Sr. García	698321475

Segunda forma normal (2FN)

Para la segunda forma normal, se necesita identificar una clave primaria y verificar que cada atributo dependa completamente de esa clave primaria. En este caso, se puede tomar el ID como clave primaria, ya que es único para cada estudiante. Sin embargo, la columna «Profesor» no depende de la clave primaria. Para corregir esto, es factible crear una tabla separada para los profesores y utilizar un ID único para cada uno.

Tabla de profesores:

ID	Profesor
1	Sr. Martínez
2	Sra. López
3	Sra. Pérez
4	Sr. García

Segunda forma normal (2FN)

Del mismo modo, en la tabla de estudiantes, se reemplaza la columna «Profesor» con el ID del profesor correspondiente para que la tabla se encuentre en 2FN:

ID	Nombre	Edad	Grado	ID Profesor	Teléfono
1	Juan	15	10	1	658253524
2	María	14	9	2	665372145
3	Carlos	16	11	3	687253940
4	Ana	15	10	4	698321475

Tercera Forma Normal (3FN)

Con la tercera forma normal, se trata de asegurar que no haya dependencias transitivas. La columna «Teléfono» depende del estudiante, pero podría depender de la tabla de profesores si cada estudiante tiene al mismo profesor. Para corregir esto, se puede mover la columna «Teléfono» a la tabla de profesores:

Tabla de profesores (actualizada):

ID	Profesor	Teléfono
1	Sr. Martínez	658253524
2	Sra. López	665372145
3	Sra. Pérez	687253940
4	Sr. García	698321475

Y la tabla de estudiantes se mantiene igual:

ID	Nombre	Edad	Grado	ID Profesor
1	Juan	15	10	1
2	María	14	9	2
3	Carlos	16	11	3
4	Ana	15	10	4

Existen mas formas normales

Forma Normal de Boyce-Codd (BCNF)

Se trata de una versión más estricta de 3NF que aborda anomalías adicionales. En este nivel de normalización, cada determinante es una clave candidata.

Cuarta Forma Normal (4NF)

Se trata de un nivel de normalización que se basa en BCNF al tratar las dependencias multivaluadas.

Quinta forma normal (5NF)

5NF es el nivel de normalización más alto que aborda las dependencias de unión. Se utiliza en escenarios específicos para minimizar aún más la redundancia dividiendo una tabla en tablas más pequeñas.

SQL

SQL - DDL: Construyendo la Estructura

Lenguaje de Definición de Datos (DDL)

- **CREATE TABLE** (Crear)
- **ALTER TABLE** (Modificar)
- **DROP TABLE** (Eliminar)

Restricciones (Constraints):

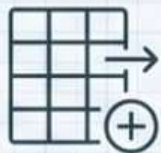
- NOT NULL, UNIQUE, CHECK
- PRIMARY KEY / FOREIGN KEY

Vistas (Views): Tablas virtuales para seguridad y simplicidad.

Código DDL

```
CREATE TABLE Empleados (  
    ID INT PRIMARY KEY,  
    Nombre VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Depto_ID INT REFERENCES Deptos(ID)  
);
```

SQL - DML: Manipulación y Gestión



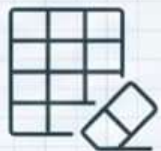
INSERT

Carga de nuevos datos en las tablas.



UPDATE

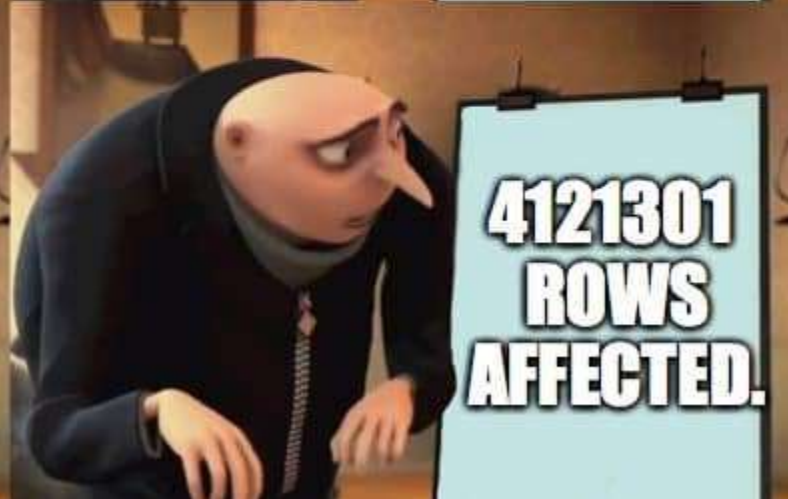
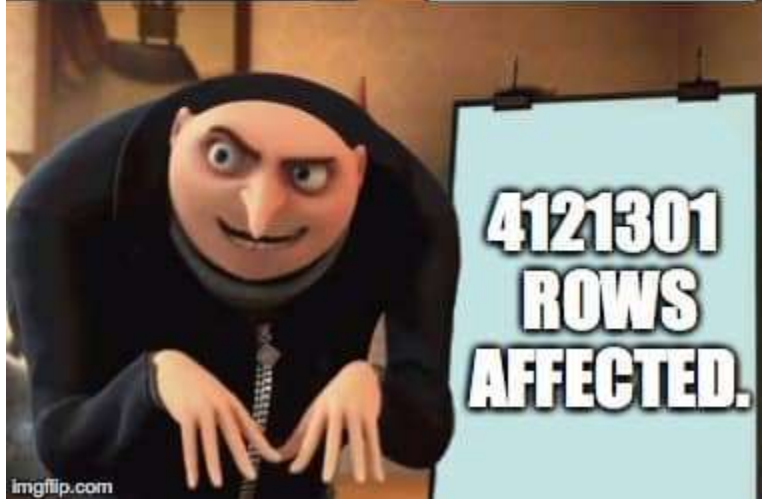
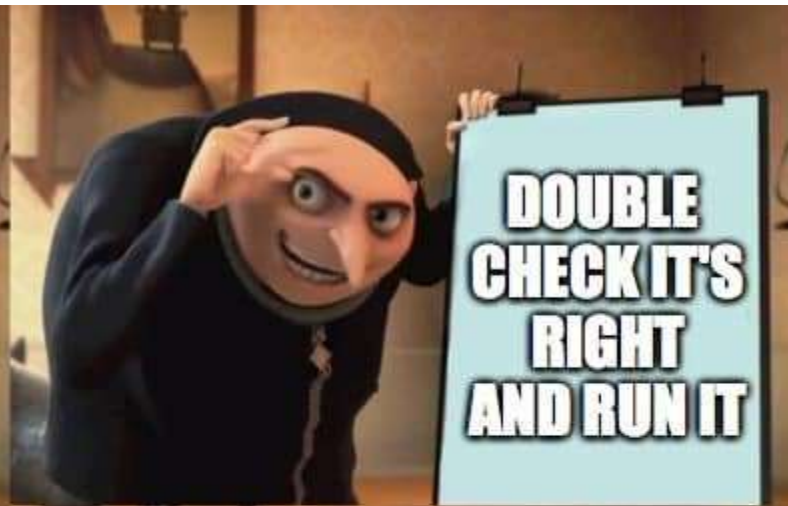
Modificación de datos existentes.



DELETE

Eliminación de registros.

Estas operaciones alteran el estado de la base de datos y están sujetas a las reglas de integridad del DDL.



SQL - La Consulta: Anatomía del SELECT



SELECT: Especifica qué columnas recuperar.

```
SELECT departamento, COUNT(*)  
FROM empleados  
WHERE salario > 50000  
GROUP BY departamento  
HAVING COUNT(*) > 5  
ORDER BY departamento;
```

→ **WHERE:** Filtrado de filas (antes de agrupar).

→ **GROUP BY:** Agrupamiento de valores comunes.

→ **HAVING:** Filtrado de grupos (después de agrupar).

→ **ORDER BY:** Ordenamiento de resultados.

Orden de evaluación de las consultas

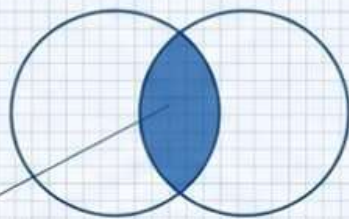
El orden lógico de evaluación de una consulta SQL no es el mismo que el orden en que la escribimos.

El motor sigue una secuencia definida:

1. **FROM** (incluye los **JOINS**) → se arman las combinaciones de filas.
2. **WHERE** → se filtran las filas individuales (antes de agrupar).
3. **GROUP BY** → se forman los grupos con las filas que sobrevivieron al WHERE.
4. **HAVING** → se filtran los grupos resultantes.
5. **SELECT** → se proyectan las columnas o agregados.
6. **DISTINCT** — si aplica, elimina duplicados del resultado proyectado.
7. **ORDER BY** → se ordena el resultado final.
8. **LIMIT** → se recorta la cantidad de filas devueltas.

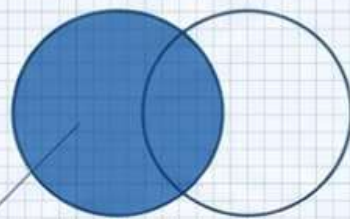
SQL - Joins: Combinando Relaciones

INNER JOIN



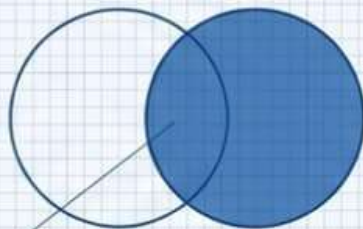
Coincidencias exactas en ambas tablas.

LEFT JOIN



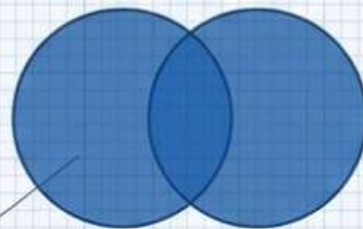
Prioridad tabla izquierda
(coincidencias + no coincidencias).

RIGHT JOIN



Prioridad tabla derecha.

FULL JOIN



Combina todo (Unión de conjuntos).

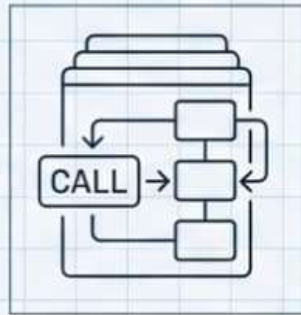
Lógica en el Servidor: Automatización y Modularidad



EVENTO: INSERT,
UPDATE, DELETE

Triggers (Disparadores)

- Acciones automáticas ante eventos (INSERT, UPDATE, DELETE). Son mecanismos que se ejecutan en respuesta a cambios específicos en los datos.
- USOS:
 - Auditoría, validaciones complejas.



Procedimientos Almacenados

- Bloques de código con nombre, ejecutados bajo demanda (CALL). Rutinas precompiladas almacenadas en la base de datos.
- VENTAJAS:
 - Modularización, reducción de tráfico de red.

Transacciones

Problemática

A menudo, desde el punto de vista del usuario de un sistema se considera a un conjunto de operaciones relacionadas como una única operación.

Por ejemplo, una transferencia de fondos desde una cuenta corriente a una cuenta de ahorros es una operación simple desde el punto de vista del cliente; sin embargo, en el sistema bancario está compuesta internamente por varias operaciones.

Evidentemente es esencial que tengan lugar **todas las operaciones** o que, en caso de fallo, **ninguna de ellas** se produzca. Sería inaceptable debitar el monto de la transferencia en la cuenta origen y que no se acredite en la cuenta de destino.

¿Qué es una transacción?

Transacción: secuencia de instrucciones (sentencias SQL) relacionadas, que deben ser tratadas como una unidad indivisible.

Ejemplos: transferencia de una cuenta a otra, la venta de un pasaje, el cambio de comisión de un alumno, etc.

Para asegurar la integridad de los datos se necesita que el sistema de base de datos mantenga las propiedades ACID de las transacciones: *atomicidad*, *consistencia*, *aislamiento (isolation)*, *durabilidad*.

Estas propiedades permiten:

- Realizar control de concurrencia.
- Realizar recuperación ante fallos.

ACID

- **Atomicidad**: la transacción se trata como una unidad atómica, se ejecutan **todas** las operaciones que contiene o no se ejecuta **ninguna**.
- **Consistencia**: la ejecución aislada de la transacción (es decir, sin otra transacción que se ejecute concurrentemente) **conserva la consistencia de la base de datos**.
- **Aislamiento** (isolation): Aunque se ejecuten varias transacciones concurrentemente, el sistema garantiza que para cada par de transacciones T_i y T_j , se cumple que para los efectos de T_i , o bien T_j ha terminado su ejecución antes de que comience T_i , o bien que T_j ha comenzado su ejecución después de que T_i termine. De este modo, **cada transacción ignora al resto de las transacciones que se ejecuten concurrentemente en el sistema**.
 - T no muestra los cambios hasta que finaliza
 - T_i nunca debe ver las fases intermedias de otra T_j
- **Durabilidad**: Tras la finalización con éxito de una transacción, los cambios realizados en la base de datos permanecen, incluso si hay fallos en el sistema.

La próxima clase veremos
concurrency y sus problemas

ACID en ejemplo

Ejemplo simplificado: consideremos que en un sistema bancario se implementa una transacción para realizar la operación “transferencia de una cuenta a otra”.

En la transacción se utilizan dos operaciones a la base de datos:

- leer(X): que lee el dato X de la base de datos
- escribir(X): que escribe el dato X en la base de datos

Sea T_i una transacción para transferir \$500 de la cuenta A a la cuenta B:

```
 $T_i$ :    leer(A);  
         A := A - 500;  
         escribir(A);  
         leer(B);  
         B := B + 500;  
         escribir(B);
```

ACID en el ejemplo

Atomicidad: Todas las operaciones de la transacción deben tratarse como una única operación. El sistema de base de datos mantiene los valores antiguos (en disco) de aquellos datos sobre los que una transacción realiza una escritura y, si la transacción no completa su ejecución, los valores antiguos se recuperan para que parezca que la transacción no se ha ejecutado (lo veremos más adelante).

Consistencia: se necesita que en la ejecución de la transacción se ejecute la resta en A y la suma en B (ambas operaciones). De esa forma se garantiza que el estado de consistencia en la BD existe antes y después de la ejecución de la transacción, sino se podría crear o destruir plata: la base de datos es inconsistente temporalmente durante la ejecución de la transacción para transferir fondos de la cuenta A a la cuenta B, con el total deducido escrito ya en A y el total incrementado todavía sin escribir en B. **El estado de la BD es consistente luego de ejecutar las operaciones de la transacción.**

ACID en el ejemplo

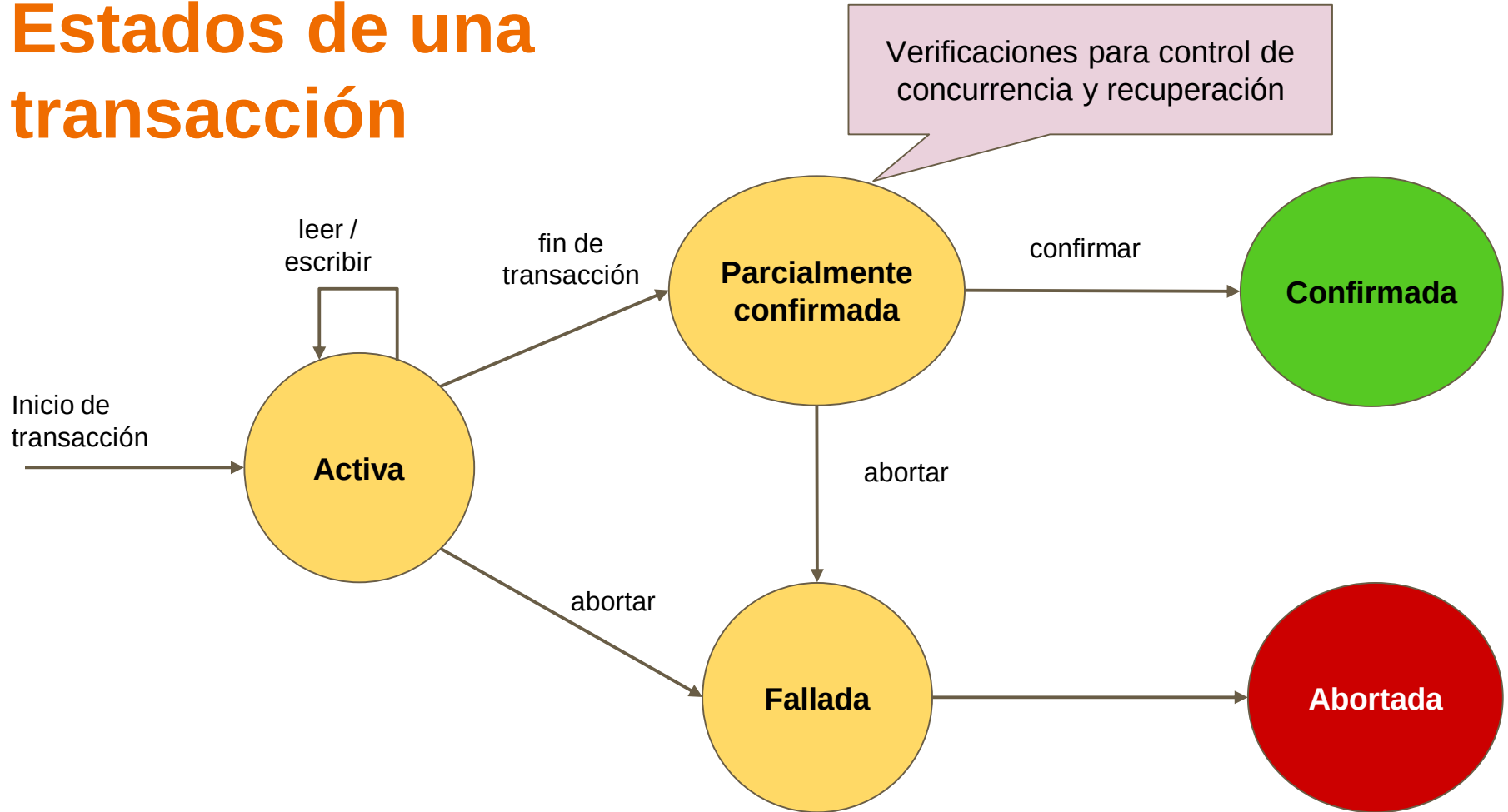
Aislamiento: aunque se aseguren las propiedades de consistencia y de atomicidad para cada transacción, si varias transacciones se ejecutan concurrentemente, se pueden entrelazar sus operaciones de un modo no deseado, produciendo un estado inconsistente. Como vimos, la BD tiene temporalmente un estado inconsistente durante la ejecución de la transacción que transfiere fondos de A a B, si una segunda transacción se ejecuta de forma concurrente lee los valores de A y B en este punto intermedio trabajará sobre valores inconsistentes y la BD permanecerá en un estado inconsistente luego de que terminen de ejecutarse las transacciones. **La propiedad de aislamiento asegura que el resultado obtenido al ejecutar concurrentemente las transacciones es un estado del sistema equivalente a uno obtenido al ejecutar una tras otra en algún orden.**

Durabilidad: una vez que se completa con éxito una transacción, persisten todas las modificaciones realizadas en la base de datos, incluso si hay un fallo en el sistema después de completarse la ejecución de dicha transacción. **Las modificaciones realizadas por la transacción se guardan en disco antes de que finalice la transacción.**

Estados de una transacción

- **Activa:** Estado inicial donde se ejecutan las operaciones de lectura y escritura. La transacción está en proceso.
- **Parcialmente confirmada:** La transacción ha completado todas sus operaciones necesarias, pero los cambios aún no se han guardado permanentemente en la base de datos principal.
- **Confirmada:** La transacción se ha completado con éxito y todos sus cambios se han guardado de forma permanente en la base de datos. Es el estado final de una transacción exitosa.
- **Fallida:** Se produce un error durante la ejecución de la transacción. En este punto, todas las operaciones realizadas hasta el momento se descartan para mantener la coherencia de la base de datos.
- **Abortada:** después del retroceso (**roll back**) de la transacción y de haber restablecido la base de datos a su estado anterior al comienzo de la transacción.

Estados de una transacción



Transacciones abortadas

Después de que una transacción es abortada, es posible:

Reiniciar la transacción: solamente si la transacción fue abortada como consecuencia de alguna *falla de hardware o algún error de software externo* (es decir, no fue causado por la lógica interna de la transacción). Una transacción reiniciada se considera una nueva transacción.

Matar la transacción: generalmente cuando ocurre un *error de lógica interno* que debe ser corregido sobrescribiendo el programa de aplicación, o cuando la entrada fue incorrecta, o cuando los datos deseados no fueron encontrados.

Espacios de datos

Almacenamiento
secundario (disco)



Input/Output

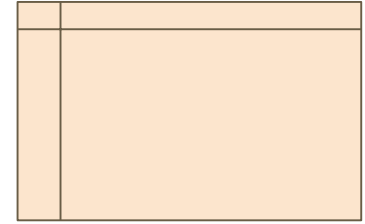


**Buffer de Datos
Compartido**

MEMORIA RAM



Buffer local transacción T_i



read/write



Buffer local transacción T_j



Transacciones en PostgreSQL

- Postgres es famoso por ser extremadamente estricto con el cumplimiento de ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad). Es su característica más fuerte para garantizar que los datos nunca se corrompan.
- Implementa bloqueo a nivel de fila, lo que permite alta concurrencia.
- Tiene un mecanismo automático de recuperación
- Tiene un sistema de detección automática de deadlocks. Si dos transacciones se bloquean mutuamente (Transacción A espera a B, y B espera a A), Postgres matará una de las dos (lanzará un error) para dejar pasar a la otra después de un tiempo (deadlock_timeout).

Transacciones en PostgreSQL - Sintaxis y

uso

- La sentencia **BEGIN**; comienza una transacción.
- La sentencia **COMMIT**; termina la transacción y almacena todos los cambios.
- Las sentencias SQL ejecutadas entre BEGIN y COMMIT se ejecutan atómicamente.
- La sentencia **ROLLBACK**; puede utilizarse para retroceder la transacción y volver al estado previo al comienzo de la misma.